

NOP Chaos Flux

使用 AI 进行大规模框架开发的实践经验

AI 大规模开发的本质：动力系统的受控收敛

canonical | 2026 年 5 月

作者简介

 毕业于清华大学工程物理系，拥有 20 多年软件架构设计经验

 致力于将物理学思想体系引入软件工程，原创性地提出可逆计算理论

 NOP 低代码平台作者，独立设计并实现后端 `nop-entropy` 与前端 `nop-chaos`

 本次分享基于 `nop-chaos-flux` 的真实开发实践—— NOP 体系内对 AMIS 方案的下一代升级替代

nop-chaos-flux 是什么



一个 **Schema 驱动的前端运行时和渲染框架**

"Schema that executes, not just renders"——Schema 不仅描述 UI，还参与执行



为 NOP 低代码体系提供下一代前端运行时基础设施

不是应用框架，不是 UI 组件库



围绕 **七个原语**构建：Template、ScopeRef、Value、Resource、Reaction、Capability、Host Projection

表单、表格、对话框、设计器、验证等都是这七个原语之上的派生系统



在 NOP 体系中的位置：`nop-chaos` 过去基于 AMIS，Flux 是脱离 AMIS 的独立重写

项目简介：技术栈与模块结构

技术栈

React 19

UI Framework

TypeScript

6

Language

Zustand

State

Vite 8

Build

Vitest 4

Test

Tailwind

4

CSS

27 个 workspace packages

→ 核心流水线

flux-core → flux-formula → flux-compiler →

flux-action-core → flux-runtime → flux-react



渲染器族 + 设计器/编辑器 + 共享基础设施 + 诊断工具

项目规模与开发数据

50

开发天数（48 天活跃）

1,207

TypeScript 文件

176K

有效代码行

27

Workspace Packages

102K

非测试 Source Code

80K

测试代码（ratio 0.79）

922

Markdown 文档

1,601

总 Commits

口径说明：以上数据均采用 **2026-05-03 冻结快照**。LOC 为文件总行数，"有效代码"为去空行和纯注释后的行数。

高速生成下架结构持续收敛：效率只是副产品

稳定性的真正证据

 边界越来越明晰

`flux-compiler` / `flux-action-core` / `flux-runtime` 三

层分离

 结构越来越纯净

`CompiledSchemaNode` 等过渡结构被清除

 验证越来越严格

lint/check 持续收紧，deep audit 多轮过滤

效率数据（在稳定的前提下）

1

开发者 + AI

7 周

开发周期

8-16x

效率倍数（保守估计）

传统估计：4-6 名高级前端工程师，3-4 个月 =
12-24 人月

真正罕见的不是快，而是在这种快里系统没有退化。

核心问题：AI 生成是扩张力，没有收敛机制就会持续漂移

⚡ 高带宽生成

AI 能在极短时间内产出大量代码、测试、文档

😊 乐观偏差

AI 会流畅地报告"已经完成"，但行为根本没落地

🔄 上下文漂移

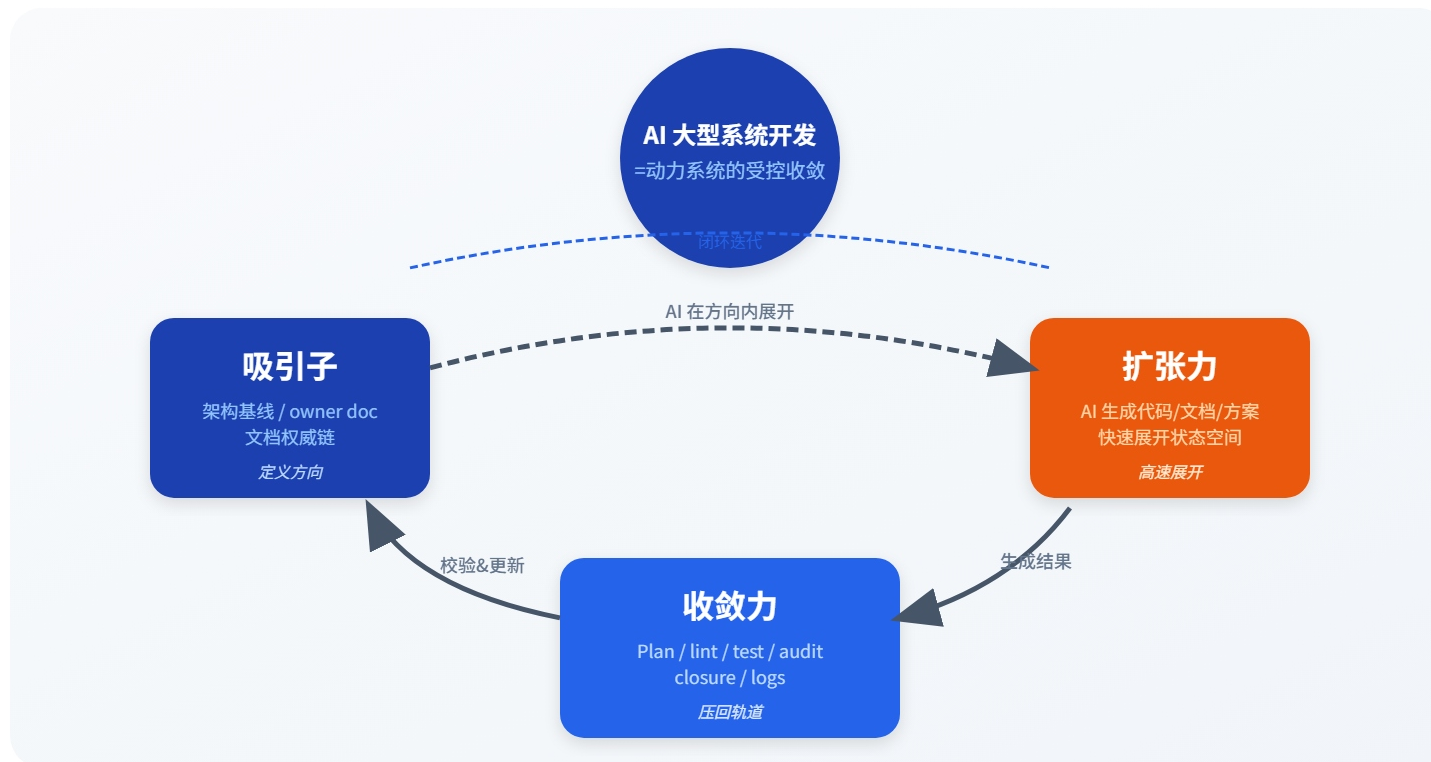
旧前提在长上下文里不断被复用，整组文件一起偏离真实基线

最危险的地方：不是"写错一段代码"，而是**局部都像对的，但整体在持续偏离。**

仓库实例：Plan 76 尝试移除本地状态镜像，结果暴露 11 个测试失败——测试也绑定了旧实现时序。

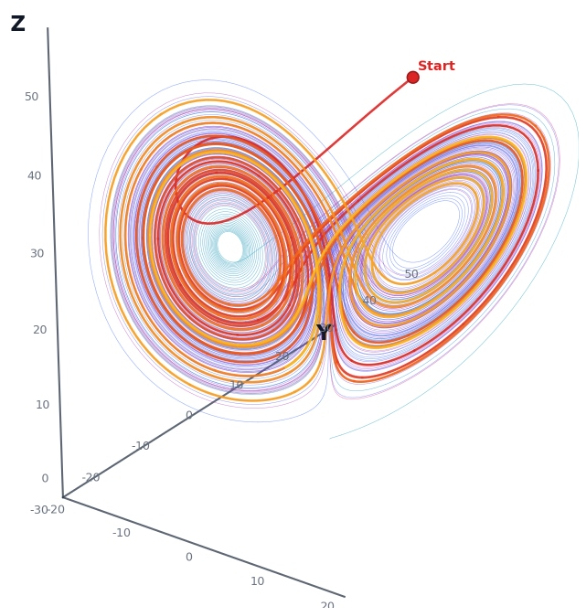
AI 的自治叙事，恰恰是整体漂移最好的掩护。

AI 时代需要新的工程隐喻：动力系统的受控收敛



没有吸引子，AI 不是在开发系统，而是在解空间中盲目扩张。

什么是动力系统，什么是吸引子



$\frac{dx}{dt} = \sigma * (y - x)$
 $\frac{dy}{dt} = x * (\rho - z) - y$
 $\frac{dz}{dt} = x * y - \beta * z$

$\sigma = 10$
 $\rho = 28$
 $\beta = 8/3$

Lorenz Attractor
Chaotic Attractor

数值积分计算 · 40,000 步轨迹

动力系统

状态随时间演化、且下一步依赖当前状态的系统。我们关心的是**轨迹**——从哪里出发，往哪里演化。

吸引子

状态空间中的一个稳定区域——从足够近的地方出发的轨迹，最终会被它吸引并长期停留在它附近。

映射到 AI 开发



状态空间 = 所有合法实现状态的集合



吸引子 = 架构基线允许的结构空间



轨迹 = 每一轮 AI 生成 + 收敛的演化路径

混沌不等于随机；局部不可预测，不等于整体失控。

仓库不再是认知的表达，而是系统真相本身

传统开发

仓库 = 认知的外化

作者认知持有系统模型

仓库主要是表达媒介



系统状态的主要载体转移

AI 深度参与后

仓库 = 系统真相的载体

代码 / diff / logs / tests / docs / audit

人：观察 · 约束 · 校正

仓库产物的新角色：

- test / lint / audit = 测量
- owner doc / plan / closure = 约束
- logs / history / bugs = 轨迹记录

当作者的连续脑内模型不再构成唯一权威背景，验收就不能依赖生成上下文本身，只能回到仓库里的外部证据重新判断。

动力系统模型与传统开发的关键区别

瀑布

强蓝图 / 长流水线

- 先设计，再实现，再测试
- 蓝图优先，阶段分明
- 默认人类可同步把关

敏捷

弱蓝图 / 短流水线

- 迭代设计，快速反馈
- 设计与实现交替推进
- 默认团队仍能持续跟上

AI 深度参与

动力系统 / 轨迹控制

- 生成速度超出人脑自然收敛
- 验收不再依赖生成上下文自证
- 重点：系统轨迹控制与受控收敛

按图施工 → 迭代施工 → 控制系统轨迹

再用流水线思维管理 AI 高速生成，只会把混乱组织得更整齐。

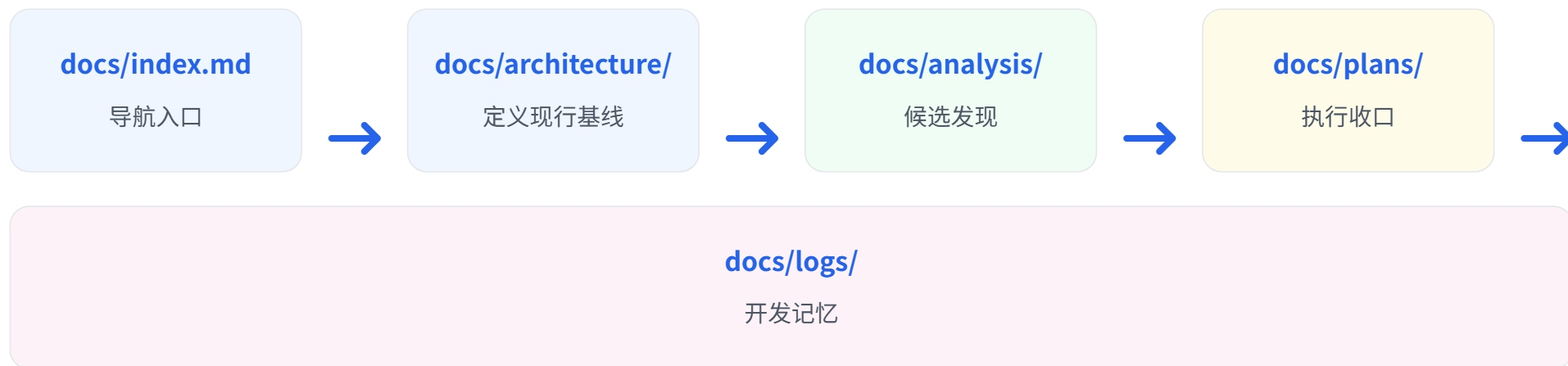
吸引子的工程实现 1：架构基线与文档权威链

U 吸引子 = 系统长期稳定收敛的结构区域

由七个原语闭合集、编译优先、数据/能力正交、统一渲染器合约、职责分层等基线共同定义

D 固定在 `docs/architecture/` 的 owner doc 中，决定每次扩张的合法方向

文档权威链



案例： `CompiledSchemaNode` 被移除，因为它已经不属于模板/实例分离的新基线。

架构的价值不在于描述一切，而在于让错误结构无法合法存在。

吸引子的工程实现 2：保留澄清过程

专门保留 `docs/discussions/`

不只写最终结论，也保留澄清过程：原始表达 → AI 复述与提问 → 用户补充 → 纠正记录 → 最终总结

后续 AI 不用凭空猜"为什么这样设计"，也能看到"为什么**不能**那样设计"

对复杂系统来说，**早期理解偏差往往比编码错误更致命**

如果只保留最后结论，后来的 AI 看不到哪些前提曾经被证伪过，哪些术语是为了消歧才改的——**吸引子就会弱化**。

扩张力：AI 负责展开，不负责裁决

AI 负责

✓ 代码生成

✓ 文档整理

✓ 方案展开

✓ 审计候选生成

不交给 AI 自主决定

✗ 架构裁决

✗ 概念发明

✗ 边界重定义

✗ 方案淘汰

核心原则：不让同一个上下文既做实现又做收敛裁判。用多个独立 session 交叉验证，人校准收敛标准。

收敛力 1: Plan 为每一次扩张定义结果面与收口条件

🎯 todo list 跟踪当前任务，Plan 定义本轮结果面与关闭条件

前者服务于当前会话的工作记忆；后者规定这轮扩张要收口到哪里、什么才算真的完成、完成必须拿什么证据证明

☰ 每个 Plan 至少写清

- Current Baseline
- Goals / Non-Goals
- Exit Criteria
- Validation Checklist
- Closure Audit Evidence

📌 硬规则：写计划前先核对 **live repo**，不沿用旧计划和旧 completion note

Plan 不是待办列表；它定义的是这一轮扩张的结果面、退出条件和独立 closure 依据。

Plan 作为收敛力的实际效果

漂移模式（Plan 机制之前）

- ⚠️ 计划过宽，没人知道什么时候算完成
- ⚠️ 只记录最后 landing 的内容，不回看整份 plan
- ⚠️ 接口和类型出现了，就误判语义已经落地
- ⚠️ 实现者自己把 plan 标成 completed

制度化的解决

- ✅ scope 太宽 → 拆 successor plan
- ✅ 接口存在 ≠ 行为落地 → closure audit 区分
- ✅ `completed` 必须来自独立 closure audit

真实案例：Plan 145 在 closure/audit 后将新跟进面分离为 Plan 146——不同结果面混在一起只会制造假关闭。

真实案例：Plan 143 的 closure assumption 被独立审计连续推翻，直到 2026-04-27 live repo 全部过线才标记 completed。

收敛力 2：开发日志作为外部化工作记忆

日志最少记录四件事

 改了什么

 为什么这么改

 相关代码/文档/计划路径

 下一步是什么

隐藏价值：保留"为什么不这么做"

- 某个收敛尝试回滚了，因为暴露了另一个 runtime regression
- 某个架构方向先只收敛文档语义，不强推 runtime 重写
- 某个能力先作为 docs-only alignment 落地，不扩大 scope

如果这些信息不进仓库，后续 AI 容易重走同样的失败路线—— **记忆缺失会让吸引力变弱。**

日志不是收尾材料，而是系统记忆。

验证体系：三重验证手段构成吸引盆地



梯度（软改进）—— 标记退化信号，不阻断当前执行，可延后集中处理

warn 级 lint 规则



势垒（硬阻断）—— 越界即失败，不可逾越

- `check:react19` — 阻断旧 API
- `check:src-artifacts` — 阻断构建产物污染
- `check:oversized-code-files` — 阻断超大文件
- `check:il8n-keys` — 阻断硬编码字符串
- `lint` — 包括 `react-hooks/exhaustive-deps`、`react-compiler` 等 error 级规则



智能审计—— 覆盖自动化手段无法捕获的语义漂移与结构偏差

deep audit、closure audit，既有硬阻断也有软改进

能靠机器提前失败的，就不要留给人类事后理解。

审计闭环：从多 Agent 发散到共识收敛



实际数据：2026-04-17 全量审计 28 项初审判定中，经独立复核后 11 项被归为非问题，**假阳性率约 39%**。

🚫 审计终态只允许两种：**"已解决"** 或 **"明确无需行动"**。禁止"延后""待跟进"等模糊终态。

🔗 高信号审计发现会逐步下沉为 lint 规则、check 脚本或测试守卫——把人工收敛力转化为自动化收敛力。

高质量审计，不是堆问题，而是尽快排除假问题。

长期收敛力：Debugger、Bug 记忆与完成判定

Bug 记忆

保存的不是 diff，而是**排查路径**：为什么难定位、先看了什么、排除了哪些假设、最后依据什么确认了问题

nop-debugger

按"AI 可消费"设计：以 live `cid` 为中心，暴露 automation-facing API

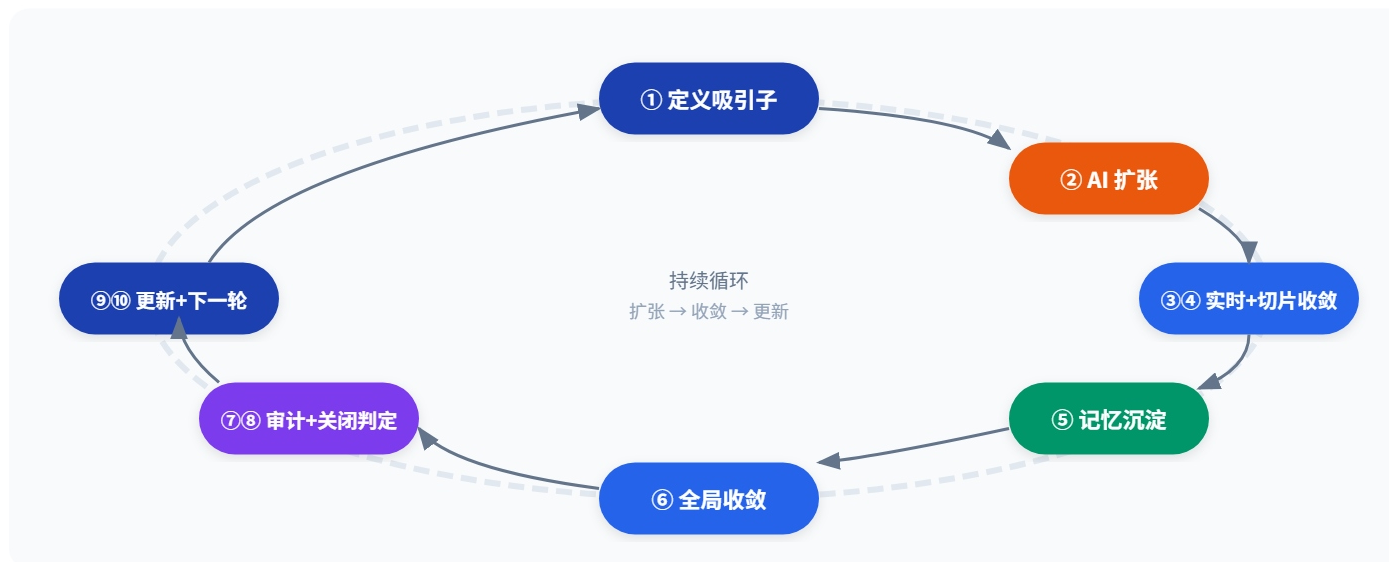
```
inspectByCid()  getSnapshot() /  
explainNodeValue()
```

完成判定

live code path 真经过 • focused/full verification 收口 • 文档同步 • leftover work 归属明确 • 独立 closure audit

`completed` 不靠声明，靠证据。

闭环如何实际运转：一个完整周期



完整周期（10步）

① 定义吸引子 → ② AI 扩张 → ③④ 局部测量与纠偏 → ⑤ 记忆沉淀 →
⑥ 全局验证轨迹是否仍落在稳定区 → ⑦⑧ 审计与关闭判定 → ⑨ 吸引子
更新 → ⑩ 下一轮

★ 典型实例：不是“修边界”，而是澄清吸引子

先定义“编译优先、职责分层”的稳定收敛方向 → AI 大量展开实现 →
复核发现系统轨迹仍未落入该稳定区 → 最终拆出 `flux-compiler` 和
`flux-action-core`，让三层职责重新收敛到既定基线

lint / check / audit / closure 是测量与纠偏手段，不是这套方法的第一性定义。

从仓库历史看这个闭环的实际轨迹

●
03-20~23

早期建模期
初始吸引子建立

●
03-25~29

首次大规模扩张
收敛力同步跟上

●
04-12~20

高密度收敛期
扩张/收敛交替最密

●
04-21~25

精确吸引子形成
三层拆分完成

结构收敛

从过渡结构到真实基线

`CompiledSchemaNode` 被消除，模板实例化成为真实基线

职责收敛

从"能跑"到"结构稳定"

`compiler / action-core / runtime` 三层拆分，系统更清晰

问题收敛

不是发现更多，而是让真实问题留下来
通过独立复核排除伪问题，只保留 live repo 中仍成立的偏离信号

50 天真正了不起的不是产出快，而是在这种速度下，架构越来越清晰。

吸引子为什么能起作用—— 解决 AI 的三个核心偏差

乐观偏差

"看起来做完了"



防止假完成

Exit Criteria + Closure Audit

验证行为闭环

上下文漂移

"旧前提不断被复用"



防止失忆

日志 + 文档权威链

恢复正确基线

局部正确 / 全局漂移

"在错误层做了正确修复"



防止方向错误

架构基线 + owner doc

限制错误方向

重点不是让 AI 不犯错，而是让错误不能长期存活。

这不是“更严的边界 + 更多审计”

AI 大规模开发首先是动力系统问题，而不是治理强化问题。

边界 / 护栏模型

🚫 先回答：**什么不能做**
核心是定义禁区、守住红线，防止系统越线

⚠️ 错误主要表现为：**越线**
判断重点是哪里违规、哪里破坏约束

✂️ `plan / lint / audit` 的作用是：**防违规、守红线**
即使没有定义长期收敛方向，这套机制也能局部运转

吸引子模型

U 先回答：**系统应向哪里长期收敛**
先定义合法稳定区，再谈什么算偏离、如何纠偏

📡 错误主要表现为：**整体漂移**
重点不是某一步是否局部正确，而是系统轨迹是否持续落入吸引子附近

📡 `plan / lint / audit` 的作用是：**把轨迹压回稳定区**
它们不是第一性原理；不先定义吸引子，控制就没有目标

状态空间 → 吸引子 → 轨迹 → 控制

区别不在约束更多，而在先定义长期收敛方向，再让所有约束服务于它。

可复制结构与核心结论

你不一定要照搬这个仓库的具体做法，但你很可能必须回答五个问题：

1

谁定义方向？

2

冲突时信谁？

3

怎么发现漂移？

4

怎么保留记忆？

5

怎么判定完成？

五个问题都有答案，闭环就成立；答不上来，高速推进只会加速失控。

可复制的不是表面做法，而是这套闭环本身——人先定义方向，AI 再高速收敛。

AI 会朝平均方案收敛——真正新的概念突破仍需人的判断

受训练偏置影响，AI 的收敛目标是它见过的平均解。即使要求它"创新"，给出的往往仍是当前知识体系里的主流方案，而不是真正新的结构观念。

✓ AI 最适合承担

实现 · 展开 · 补全 · 验证 · 整理
在既有知识边界内做高质量执行

⚠ 不适合单独承担

核心概念发明 · 边界重定义 · 第一性原理层面的架构突破
这些在当前条件下不会由 AI 自己长出来

案例： `ActionScope` 与 `Data Scope` 分离并采用词法作用域——这类方案不是让 AI 自由采样能自然长出来的，必须先由人定义吸引子，再让 AI 在其内收敛执行。

执行力是 AI 的强项，但新的概念发明仍然来自人的判断。

附录 A：分享中可引用的仓库材料

核心文档

[docs/index.md](#)

[docs/plans/00-plan-authoring-and-execution-guide.md](#)

[docs/discussions/README.md](#)

[docs/bugs/00-bug-fix-note-writing-guide.md](#)

分析与审计

[docs/analysis/2026-04-17-deep-audit/index.md](#)

[docs/references/maintenance-checklist.md](#)

架构文档

[docs/architecture/debugger-runtime.md](#)

[docs/articles/flux-architecture-evolution.md](#)

方法论文章

[docs/articles/ai-assisted-framework-development-methodology.md](#)

[docs/articles/ai-assisted-large-framework-best-practices.md](#)

[docs/articles/ai-native-large-system-development-lessons.md](#)

附录 B：项目数据速查表

指标	数值
Workspace packages	27
TypeScript 文件	1,207
TypeScript 总 LOC	199,769
TypeScript 有效代码	175,596
非测试 source code	102,156 (820 files)
测试 code	80,410 (422 files, ratio 0.79)
文档文件	922 个 Markdown
文档行数	230,116 LOC
总 commits	1,601 (截至 2026-05-03)
开发周期	50 天 (48 天活跃)
日均 commits	~33

附录 C：吸引子模型关键术语速查

动力学概念	软件工程对应	仓库实例
吸引子	架构基线	<code>docs/architecture/*.md</code>
扩张力	AI 生成	代码实现、方案展开
收敛力	工程机制	Plan / lint / test / closure audit
闭环	扩张 → 收敛 → 基线更新	完整开发周期
漂移	偏离真实基线	文档漂移、计划漂移
势能井	自动化验证规则	eslint error、check 脚本
记忆	跨 session 可追溯上下文	<code>docs/logs/</code> / <code>docs/bugs/</code>

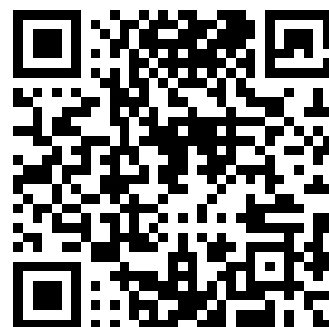
Q&A

问题与讨论

微信公众号



微信群



项目地址

GitHub: <https://github.com/entropy-cloud/nop-chaos-flux>

Gitee: <https://gitee.com/canonical-entropy/nop-chaos-flux>